

The Hubbard atom from partial spectral functions

Numerical companion to

F. B. Kugler, S.-S. B. Lee and J. von Delft, *Multipoint Correlation Functions: Spectral Representation and Numerical Evaluation*, [Phys. Rev. X **11**, 041006 \(2021\)](#)

The plan is to assemble the Matsubara correlators of the half-filled Hubbard atom directly from their spectral representation, and to check them against the closed forms the paper quotes:

Step	Content	Benchmark
1	spectrum: $H, d_\sigma, d_\sigma^\dagger, n_\sigma$; diagonalise; $\rho_i = e^{-\beta E_i} / Z$	$Z = 2(1 + e^{\beta u})$ by hand
2	partial spectral functions, Eq. (28), general in ℓ	—
3	the kernel, Eq. (46), two-branch form	finite by construction
4	$\ell = 2$: assemble Eq. (39)	$G(i\nu) = \frac{1}{2} \sum_{\pm} (i\nu \pm U/2)^{-1}$, App. E
5	$\ell = 4$: 24 permutations, subtract Eq. (73), amputate Eq. (76)	Eqs. (85a), (85b)
6	null test at $U = 0$	every connected object vanishes

All six steps are implemented and checked, and a final section repeats the calculation in the **Keldysh formalism** (Eqs. 49–69), reusing the same PSFs with only the kernel replaced. The test suite (`julia --project test/runtests.jl`) carries 52,067 assertions.

Model: $H = \varepsilon_d(n_\uparrow + n_\downarrow) + Un_\uparrow n_\downarrow$ at $\varepsilon_d = -U/2$, on the four states $|0\rangle, |\uparrow\rangle, |\downarrow\rangle, |\uparrow\downarrow\rangle$. This is the $U/\Delta \rightarrow \infty$ limit of the symmetric AIM, Eq. (79).

In [1]:

```
include(joinpath(@__DIR__, "..", "src", "HubbardAtom.jl"))
using .HubbardAtom
using LinearAlgebra
using Printf
```

Step 1 — spectrum

Build the operators with correct fermionic signs, diagonalise H , and form the Boltzmann weights. Everything later hangs off the eigenbasis produced here: Eq. (28) needs the matrix elements $(O)_{i,i+1}$ in the energy eigenbasis, which `to_eigenbasis` supplies.

The operators come from a Jordan–Wigner transform with fermionic mode order $(\uparrow, \downarrow)$,

$$d_{\uparrow} = 1 \otimes f, \quad d_{\downarrow} = f \otimes z, \quad f = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

so the parity string z over the $\uparrow$ mode is what produces $d_{\downarrow}|\uparrow\downarrow\rangle = -|\uparrow\rangle$. That sign is not cosmetic: it feeds every permutation sign ζ_p in Eq. (39).

In [2]:

```
U, β = 2.0, 5.0
model = HubbardAtomModel(U, β)
ops = operators(model)

@printf("U = %.3f, β = %.3f, εd = %.3f, u = U/2 = %.3f\n",
        model.U, model.β, level_position(model), half_interaction(model))
println("\nFock basis order: ", join(BASIS_LABELS, " "))
println("\nH (Fock basis) =")
display(ops.H)
```

U = 2.000, β = 5.000, εd = -1.000, u = U/2 = 1.000

Fock basis order:

|0⟩ |↑⟩ |↓⟩ |↑↓⟩

H (Fock basis) =

```
4x4 Matrix{Float64}:
 0.0  0.0  0.0  0.0
 0.0 -1.0  0.0  0.0
 0.0  0.0 -1.0  0.0
 0.0  0.0  0.0  0.0
```

In [3]:

```
# Fermionic algebra: the checks that catch a wrong Jordan-Wigner sign.
I4 = Matrix{Float64}(1.0I, DIM, DIM)

println("{d↑, d↑} == 1      : ", ops.d_up * ops.dag_up + ops.dag_up * ops.d_up ≈
I4)
println("{d↓, d↓} == 1      : ", ops.d_dn * ops.dag_dn + ops.dag_dn * ops.d_dn ≈
I4)
println("{d↑, d↓} == 0      : ", ops.d_up * ops.dag_dn + ops.dag_dn * ops.d_up ≈
zeros(DIM, DIM))
println("{d↑, d↓} == 0      : ", ops.d_up * ops.d_dn + ops.d_dn * ops.d_up ≈ zer
os(DIM, DIM))
println("d↑² == d↓² == 0    : ", ops.d_up^2 ≈ zeros(DIM, DIM) && ops.d_dn^2 ≈ ze
ros(DIM, DIM))
println("[H, N] == 0        : ", ops.H * ops.N ≈ ops.N * ops.H)
println("[H, Sz] == 0       : ", ops.H * ops.Sz ≈ ops.Sz * ops.H)

# the fermionic sign itself
up_dn = [0.0, 0.0, 0.0, 1.0]
up     = [0.0, 1.0, 0.0, 0.0]
println("\nd↓|↑↓⟩ = -|↑⟩      : ", ops.d_dn * up_dn ≈ -up)
println("d↑|↑↓⟩ = +|↓⟩      : ", ops.d_up * up_dn ≈ [0.0, 0.0, 1.0, 0.0])
```

```

{d↑, d↑↑} == 1      : true
{d↓, d↑↓} == 1      : true
{d↑, d↑↓} == 0      : true
{d↑, d↓} == 0       : true
d↑2 == d↓2 == 0    : true
[H, N] == 0         : true
[H, Sz] == 0        : true

```

```

d↓|↑↓⟩ = -|↑⟩      :
true
d↑|↑↓⟩ = +|↓⟩      : true

```

In [4]:

```

sp = spectrum(model)

println("eigenenergies   E = ", sp.E)
println("closed form    = ", eigenenergies_analytic(model))
println("agree          : ", sp.E ≈ eigenenergies_analytic(model))

println("\nH·V == V·diag(E) : ", ops.H * sp.V ≈ sp.V * Diagonal(sp.E))
println("V orthonormal    : ", permutedims(sp.V) * sp.V ≈ Matrix{1.0I, DIM, DIM})

```

```

eigenenergies   E =
[-1.0, -1.0, 0.0, 0.0]
closed form     =
[-1.0, -1.0, 0.0, 0.0]
agree           : true

```

```

H·V == V·diag(E) : true
V orthonormal    : true

```

In [5]:

```

# Check Z by hand. With u = U/2 and εd = -U/2 the spectrum is
#   E(|0⟩) = 0, E(|↑⟩) = E(|↓⟩) = -u, E(|↑↓⟩) = 2εd + U = 0,
# so      Z = 1 + e^{βu} + e^{βu} + 1 = 2(1 + e^{βu}).
Z_hand = partition_function_analytic(model)

@printf("Z (diagonalisation) = %.15g\n", sp.Z)
@printf("Z (by hand)         = %.15g\n", Z_hand)
@printf("relative difference = %.3e\n", abs(sp.Z - Z_hand) / Z_hand)
println("agree to machine precision: ", isapprox(sp.Z, Z_hand; rtol = 1e-12))

```

```

Z (diagonalisation) = 298.826318205153
Z (by hand)         = 298.826318205153
relative difference = 1.902e-16
agree to machine precision:
true

```

In [6]:

```

# Boltzmann weights and the quantum numbers of each eigenstate.
Nvals, Szvals = quantum_numbers(sp, ops)

println(" i      E_i      p_i      N      Sz")
println("-"^50)
for i in 1:DIM
    @printf("%2d %9.5f %11.4e %5.1f %6.1f\n", i, sp.E[i], sp.p[i], Nvals[i], Szvals[i])
end
@printf("\nΣ p_i = %.15g\n", sum(sp.p))

```

i	E_i	ρ_i	N	Sz
1	-1.00000	4.9665e-01	1.0	-0.5
2	-1.00000	4.9665e-01	1.0	0.5
3	0.00000	3.3464e-03	0.0	0.0
4	0.00000	3.3464e-03	2.0	0.0

$\sum \rho_i = 1$

In [7]:

```
# Particle-hole symmetry at  $\epsilon_d = -U/2$  pins the atom to half filling.
@printf("<n↑>      = %.15g\n", thermal_average(sp, ops.n_up))
@printf("<n↓>      = %.15g\n", thermal_average(sp, ops.n_dn))
@printf("<N>        = %.15g\n", thermal_average(sp, ops.N))
@printf("<Sz>       = %.3e\n",  thermal_average(sp, ops.Sz))
@printf("<n↑n↓>     = %.15g   (exact: 1/Z = %.15g)\n",
        thermal_average(sp, ops.n_up * ops.n_dn), 1 / sp.Z)
@printf("<H>        = %.15g\n", thermal_average(sp, ops.H))

# tanh( $\beta u/2$ ) of Eq. (85) is this spectrum's polarisation
u = half_interaction(model)
singly = 2 * exp( $\beta * u$ ) / Z_hand
@printf("\n2·P(singly occupied) - 1 = %.15g\tanh( $\beta u/2$ )           = %.15g\n",
        2 * singly - 1, tanh( $\beta * u / 2$ ))
```

```
<n↑>      = 0.5
<n↓>      = 0.5
<N>       = 1
<Sz>      = 0.000e+00
<n↑n↓>    = 0.00334642546214243   (exact: 1/Z = 0.00334642546214243)
<H>       = -0.993307149075715
2·P(singly occupied) - 1 = 0.98661429815143
tanh( $\beta u/2$ )      = 0.98661429815143
```

In [8]:

```
# What Step 2 will consume: operators in the energy eigenbasis, Eq. (28).
d_rot = to_eigenbasis(sp, ops.d_up)
println("d↑ in the energy eigenbasis:")
display(round.(d_rot, digits = 12))
println("\nnonzero elements (i, j, value), each lowering N by one:")
for j in 1:DIM, i in 1:DIM
    if abs(d_rot[i, j]) > 1e-10
        @printf("  (%d, %d)  % .6f    N: %.1f → %.1f\n",
                i, j, d_rot[i, j], Nvals[j], Nvals[i])
    end
end
```

```
d↑ in the energy eigenbasis:
4×4 Matrix{Float64}:
 0.0  0.0  0.0  1.0
 0.0  0.0  0.0  0.0
 0.0  1.0  0.0  0.0
 0.0  0.0  0.0  0.0
nonzero elements (i, j, value), each lowering N by one:
 (3, 2)  1.000000    N: 1.0 → 0.0
 (1, 4)  1.000000    N: 2.0 → 1.0
```

In [9]:

```
# Z against its closed form across a parameter sweep, including  $U = 0$  and  $U < 0$ .
println("    U       $\beta$       Z (diag)      Z (hand)      rel.err")
println("-"^66)
```

```

for (Ui, βi) in [(1.0, 1.0), (2.0, 5.0), (4.0, 0.3), (0.0, 2.0), (8.0, 10.0), (-2.0, 1.5)]
    mi = HubbardAtomModel(Ui, βi)
    spi = spectrum(mi)
    Zi = partition_function_analytic(mi)
    @printf("%6.1f %6.1f %16.8e %16.8e %10.2e\n",
           Ui, βi, spi.Z, Zi, abs(spi.Z - Zi) / Zi)
end

```

U	β	Z (diag)	Z (hand)	rel.err
1.0	1.0	5.29744254e+00	5.29744254e+00	1.68e-16
2.0	5.0	2.98826318e+02	2.98826318e+02	1.90e-16
4.0	0.3	5.64423760e+00	5.64423760e+00	0.00e+00
0.0	2.0	4.00000000e+00	4.00000000e+00	0.00e+00
8.0	10.0	4.70770534e+17	4.70770534e+17	0.00e+00
-2.0	1.5	2.44626032e+00	2.44626032e+00	0.00e+00

Step 2 — partial spectral functions

Eq. (28), implemented literally and kept general in ℓ , so the one function serves $\ell = 2$ (Step 4), $\ell = 3$ and $\ell = 4$ (Step 5):

$$S[\mathcal{O}_p](\omega'_p) = \sum_{1 \dots \ell} \rho_1 \left[\prod_{i=1}^{\ell-1} (O_{\bar{i}})_{i,i+1} \delta(\omega'_{1 \dots \bar{i}} - E_{i+1,1}) \right] (O_{\bar{\ell}})_{\ell,1},$$

$$E_{i+1,1} = E_{i+1} - E_1.$$

`psf` returns one `PSFTerm` per eigenstate cycle with nonvanishing weight, each carrying

- **position** — the $\ell - 1$ partial sums $\omega'_{1 \dots \bar{i}} = E_{i+1} - E_1$ at which the deltas sit. These are precisely the arguments the Eq. (46) kernel consumes, through $\Omega_{1 \dots \bar{i}} = i\omega_{1 \dots \bar{i}} - \omega'_{1 \dots \bar{i}}$.
- **weight** — $\rho_1 \prod_{i=1}^{\ell} (O_{\bar{i}})_{i,i+1}$, the product taken *cyclically* so the last factor is $(O_{\bar{\ell}})_{\ell,1}$. This is exactly the numerator of Eq. (29), which is what Eqs. (26)–(28) collapse to.

Operators are handed over in the permuted order $\mathcal{O}_p = (O_{\bar{1}}, \dots, O_{\bar{\ell}})$, so one call covers one permutation p ; Steps 4 and 5 loop over permutations and pair each **position** with the matching $i\omega_{1 \dots \bar{i}}$.

Because the PSFs are sums of δ 's, the ω' integral in Eq. (39) will be a **finite sum** over these terms — no grids, no broadening.

In [10]:

```

# ℓ = 2, the simplest case: S[d↑, d↑†].
terms2 = psf(sp, ops.d_up, ops.dag_up)
println("raw terms: ", length(terms2), " (one per eigenstate cycle with nonzero weight)\n")
println("      ω'      weight      eigenstate cycle")
println("-" ^ 50)
for t in terms2
    @printf("%8.3f %12.6e %s\n", t.position[1], real(t.weight), string(t.state s))
end

```

```

end

# Merging coincident deltas gives the PSF as an actual measure.
println("\naggregated:")
for t in aggregate_psf(terms2)
    @printf("%8.3f  %12.6e\n", t.position[1], real(t.weight))
end

```

raw terms: 2
(one per eigenstate cycle with nonzero weight)

ω'	weight	eigenstate cycle
-1.000	3.346425e-03	[3, 2]
1.000	4.966536e-01	[1, 4]

aggregated:

-1.000	3.346425e-03
1.000	4.966536e-01

In [11]:

```

# Closed form for this PSF. d↑ connects |↑⟩→|0⟩ and |↑↓⟩→|↓⟩, so the deltas sit
# at  $\omega' = \mp u$  with weights  $\rho|0\rangle = 1/Z$  and  $\rho|\downarrow\rangle = e^{\{\beta u\}}/Z$ .
agg2 = aggregate_psf(terms2)
byw = Dict{round(t.position[1]; digits = 9) => real(t.weight) for t in agg2}

@printf("weight at  $\omega' = -u = %5.2f$  :  %.15g  (exact 1/Z      = %.15g)\n",
        -u, byw[round(-u; digits = 9)], 1 / sp.Z)
@printf("weight at  $\omega' = +u = %5.2f$  :  %.15g  (exact  $e^{\{\beta u\}}/Z = %.15g$ )\n",
        u, byw[round(u; digits = 9)], exp( $\beta * u$ ) / sp.Z)

```

weight at $\omega' = -u = -1.00$: 0.00334642546214243 (exact 1/Z = 0.00334642546214243)
weight at $\omega' = +u = 1.00$: 0.496653574537858 (exact $e^{\{\beta u\}}/Z = 0.496653574537857$)

In [12]:

```

# Sum rule. Summing Eq. (28) over every eigenstate cycle collapses the cyclic
# product to a trace:
#  $\sum \text{weights} = \text{tr}(\rho \, 0_{1^-} \, 0_{2^-} \dots 0_{\ell}) = \langle 0_{1^-} \, 0_{2^-} \dots 0_{\ell} \rangle$ ,
# the equal-time expectation value of the operator product in the same order.
# This holds for every  $\ell$ , and is the main check on Step 2.
labelled = [
    ("(d↑, d↑)", (ops.d_up, ops.dag_up)),
    ("(d↑, d↑)", (ops.dag_up, ops.d_up)),
    ("(n↑, n↓)", (ops.n_up, ops.n_dn)),
    ("(n↑, d↑, d↑)", (ops.n_up, ops.d_up, ops.dag_up)),
    ("(d↑, n↑, d↑)", (ops.d_up, ops.n_up, ops.dag_up)),
    ("(d↑, d↑, d↑, d↑)", (ops.d_up, ops.dag_up, ops.d_up, ops.dag_up)),
    ("(d↑, d↑, d↓, d↑)", (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn)),
    ("(d↑, d↑, d↓, d↑)", (ops.d_up, ops.dag_dn, ops.d_dn, ops.dag_up)),
]

println("  $\ell$   operator tuple          terms   $\sum \text{weights}$    $\langle 0_{1^-} \dots 0_{\ell} \rangle$   |
diff|")
println("-"^82)
for (name, Os) in labelled
    terms = psf(sp, Os)
    lhs = real(psf_total_weight(terms))
    rhs = thermal_average(sp, reduce(*, Os))
    @printf(" %d  %-22s %5d  %14.10f  %14.10f  %8.1e\n",
            length(Os), name, length(terms), lhs, rhs, abs(lhs - rhs))
end

```

ℓ	operator tuple	terms	Σ weights	$\langle 0_1 \dots 0_\ell \rangle$	diff
2	(d↑, d↑↑)	2	0.5000000000	0.5000000000	0.0e+00
2	(d↑↑, d↑)	2	0.5000000000	0.5000000000	0.0e+00
2	(n↑, n↓)	1	0.0033464255	0.0033464255	0.0e+00
3	(n↑, d↑, d↑↑)	0	0.0000000000	0.0000000000	0.0e+00
3	(d↑, n↑, d↑↑)	2	0.5000000000	0.5000000000	0.0e+00
4	(d↑, d↑↑, d↑, d↑↑)	2	0.5000000000	0.5000000000	0.0e+00
4	(d↑, d↑↑, d↓, d↑↓)	1	0.0033464255	0.0033464255	0.0e+00
4	(d↑, d↑↓, d↓, d↑↑)	1	0.4966535745	0.4966535745	0.0e+00

In [13]:

```
# A first look ahead to Step 4. The local spectral function is
#  $A(\omega) = S[d, d^\dagger](\omega) + S[d^\dagger, d](\omega)$  ,
# and Appendix E's exact propagator  $G(iv) = \frac{1}{2} \Sigma \pm (iv \pm U/2)^{-1}$  says it must be
# two poles of weight  $\frac{1}{2}$  at  $\omega = \pm u$  -- with no temperature dependence at all,
# even though each PSF separately is strongly  $\beta$  dependent.
A = Dict{Float64, Float64}()
for t in aggregate_psf(psf(sp, ops.d_up, ops.dag_up))
    k = round(t.position[1]; digits = 9)
    A[k] = get(A, k, 0.0) + real(t.weight)
end
for t in aggregate_psf(psf(sp, ops.dag_up, ops.d_up))
    k = round(-t.position[1]; digits = 9) # note the reflection  $\omega \rightarrow -\omega$ 
    A[k] = get(A, k, 0.0) + real(t.weight)
end

println("     $\omega$           A( $\omega$ )      expected")
println("-"^34)
for k in sort(collect(keys(A)))
    @printf("%8.3f  %9.6f  %9.6f\n", k, A[k], 0.5)
end
@printf("\n $\Sigma A(\omega) = %.15g$  (sum rule  $\langle \{d, d^\dagger\} \rangle = 1$ )\n", sum(values(A)))
```

ω	A(ω)	expected
-1.000	0.500000	0.500000
1.000	0.500000	0.500000

$\Sigma A(\omega) = 1$ (sum rule $\langle \{d, d^\dagger\} \rangle = 1$)

In [14]:

```
#  $\ell = 4$ : the object Step 5 will need, here for one of the 24 permutations.
Os4 = (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn)
terms4 = psf(sp, Os4)

println(" $\ell = 4$ , permutation (d↑, d↑↑, d↓, d↑↓)")
println("raw terms: ", length(terms4), " distinct positions: ", length(aggregate_psf(terms4)))
println("\n   $\omega'_1$        $\omega'_{12}$        $\omega'_{123}$       weight      cycle")
println("-"^62)
for t in terms4
    @printf("%7.3f  %7.3f  %7.3f  %12.6e  %s\n",
            t.position[1], t.position[2], t.position[3], real(t.weight), string(t.states))
end

# The individual frequencies behind those partial sums conserve energy.
for t in terms4
     $\omega$  = psf_frequencies(t)
    @printf("\n $\omega' = %s$ \n  $\Sigma \omega' = %.2e$  (energy conservation  $\omega'_1 \dots \omega'_\ell = 0$ )\n",
            string(round( $\omega$ , digits = 6)), sum( $\omega$ ))
end
```

$\ell = 4$, permutation (d↑, d↑↑, d↓, d↑↓)
raw terms: 1 distinct positions: 1

$\omega'_{1\bar{1}}$	$\omega'_{1\bar{2}}$	$\omega'_{1\bar{2}\bar{3}}$	weight	cycle
-1.000	0.000	-1.000	3.346425e-03	[3, 2, 3, 1]

$\omega' = [-1.0, 1.0, -1.0, 1.0]$
 $\Sigma\omega' = 0.00\text{e}+00$ (energy conservation $\omega'_{1\bar{1}\bar{2}\bar{3}} = 0$)

Step 3 — the kernel

Eq. (46), the two-branch form, with an explicit test for a vanishing composite $\Omega_{1\bar{1}\dots\bar{j}}$:

$$K(\Omega_p) = \begin{cases} \prod_{i=1}^{\ell-1} \Omega_{1\bar{1}\dots\bar{i}}^{-1}, & \text{if } \prod_{i=1}^{\ell-1} \Omega_{1\bar{1}\dots\bar{i}} \neq 0, \\ -\frac{1}{2} \left[\beta + \sum_{\substack{i=1 \\ i \neq j}}^{\ell-1} \Omega_{1\bar{1}\dots\bar{i}}^{-1} \right] \prod_{\substack{i=1 \\ i \neq j}}^{\ell-1} \Omega_{1\bar{1}\dots\bar{i}}^{-1}, & \text{if } \Omega_{1\bar{1}\dots\bar{j}} = 0, \end{cases}$$

with $\Omega_{1\bar{1}\dots\bar{i}} = i\omega_{1\bar{1}\dots\bar{i}} - \omega'_{1\bar{1}\dots\bar{i}}$. Deliberately **not** Eq. (45) with an ϵ -regulator: Eq. (46) never forms a divergent factor at all — the vanishing one is *excluded* from both the sum and the product — so it is finite by construction.

One design decision. Ω vanishes only when *both* parts do: the Matsubara part $\omega_{1\bar{1}\dots\bar{j}}$ and, through a degeneracy $E_{j+1} = E_1$, the spectral part $\omega'_{1\bar{1}\dots\bar{j}}$. The Matsubara part is exactly zero or not at all, so `MatsubaraFreq` stores the **integer** m in $\omega = m\pi/\beta$ and the test is an integer comparison. Float-comparing Ω to zero would confuse a genuinely small frequency — π/β is tiny at large β — with a composite that is identically zero and needs the anomalous branch.

In [15]:

```
# Regular branch: a plain product of inverse composites.
Qreg = ComplexF64[1.5 + 0.5im, -2.0 + 1.0im, 0.25 - 3.0im]
println("regular K = ", matsubara_kernel(Qreg, [false, false, false], β))
println(" ∏ Ω⁻¹ = ", prod(inv, Qreg), "\n")

# Anomalous branch, ℓ = 2: empty product and empty sum leave K = -β/2.
println("ℓ=2 anomalous K = ", matsubara_kernel(ComplexF64[0.0], [true], β),
        " (-β/2 = ", -β/2, ") \n")

# Anomalous branch, ℓ = 4 with j = 2. This is literally the second part of
# Eq. (B26): -½[β + 1/Ω₁⁻ + 1/Ω₁⁻²³⁻] / (Ω₁⁻ Ω₁⁻²³⁻).
Qan = ComplexF64[1.5 + 0.5im, 0.0, -0.75 + 2.0im]
K = matsubara_kernel(Qan, [false, true, false], β)
B26 = -0.5 * (β + inv(Qan[1]) + inv(Qan[3])) * inv(Qan[1]) * inv(Qan[3])
println("ℓ=4 anomalous K = ", K)
println("Eq. (B26) = ", B26)
println("agree : ", K ≈ B26)
println("finite : ", isfinite(real(K)) && isfinite(imag(K)),
        " (no 1/0 is ever formed)")
```

```

regular    K = 0.005517241379310348
- 0.09379310344827586im
[]  $\Omega^{-1}$     = 0.005517241379310337 - 0.09379310344827586im

 $\ell=2$  anomalous K = -2.5 - 0.0im    ( $-\beta/2 = -2.5$ )

 $\ell=4$  anomalous K = 0.5797860761869017
+ 0.5660048789641584im
Eq. (B26)    = 0.5797860761869018 + 0.5660048789641583im
agree        : true
finite       : true    (no  $1/0$  is ever formed)

```

In [16]:

```

# Why the integer index matters. At  $\beta = 1e12$  a genuinely nonzero bosonic
# frequency is numerically smaller than any sane float tolerance, yet  $\Omega \neq 0$ 
# and the regular branch is the correct one.
 $\beta_{\text{big}} = 1e12$ 
msbig = [MatsubaraFreq(1), MatsubaraFreq(1), MatsubaraFreq(-1), MatsubaraFreq(-1)]
 $\Omega_b$ , vanb = composites(msbig, [0.0, 0.0, 0.0],  $\beta_{\text{big}}$ ; atol = 1e-10)
@printf(" $\Omega_{12}$     = %.3e    ( $|\Omega_{12}| < \text{atol} = 1e-10$ )\n", abs( $\Omega_b[2]$ ))
println("flagged as vanishing? ", vanb[2], "    <- integer test says  $m_{12} =$ ",
        msbig[1].m + msbig[2].m, "  $\neq 0$ ")
println("so the kernel takes the regular branch: ",
        matsubara_kernel( $\Omega_b$ , vanb,  $\beta_{\text{big}}$ )  $\approx$  prod(inv,  $\Omega_b$ ))

```

```

 $\Omega_{12}$     = 6.283e-12    ( $|\Omega_{12}| < \text{atol} = 1e-10$ )
flagged as vanishing? false    <- integer test says  $m_{12} = 2 \neq 0$ 
so the kernel takes the regular branch: true

```

Step 4 — first milestone, $\ell = 2$

Assemble Eq. (39),

$$G(i\omega) = \sum_p \zeta_p \int d^{\ell-1} \omega'_p K(i\omega_p - \omega'_p) S[\mathcal{O}_p](\omega'_p),$$

for $\mathcal{O} = (d_\sigma, d_\sigma^\dagger)$ and compare against the exact propagator of Appendix E,

$$G(i\nu) = \frac{1}{2} \sum_{\pm} (i\nu \pm U/2)^{-1} = \frac{-i\nu}{\nu^2 + u^2}.$$

The sign ζ_p is $+1$ (-1) if $\mathcal{O}_p$ differs from $\mathcal{O}$ by an even (odd) number of transpositions of **fermionic** operators — so it is not simply the parity of p when bosonic operators are present.

The paper's stop rule: do not proceed until this matches to machine precision.

In [17]:

```

println("  n |          G(iv) from Eq. (39)          |          exact (App. E)
| rel err")
println("-"^96)
for n in 0:5
    v = fermionic_freq(n)
    got = propagator(model, v; sp = sp, ops = ops)
    want = propagator_exact(model, v)
    @printf("%3d | %+.15f %+.15fim | %+.10f %+.10fim | %.2e\n",
            n, real(got), imag(got), real(want), imag(want), abs(got-want)/abs(want))

```

```
t))
end
```

n	G(iv) from Eq. (39)	exact (App. E)	rel err
0	+0.0000000000000000 -0.450477243368389im	+0.000000000000 -0.4504772434im	1.23e-16
1	+0.0000000000000000 -0.413997749252169im	+0.000000000000 -0.4139977493im	0.00e+00
2	+0.0000000000000000 -0.289025482222236im	+0.000000000000 -0.2890254822im	1.92e-16
3	+0.0000000000000000 -0.216188454372683im	+0.000000000000 -0.2161884544im	1.28e-16
4	+0.0000000000000000 -0.171476420146551im	+0.000000000000 -0.1714764201im	1.62e-16
5	+0.0000000000000000 -0.141719536856365im	+0.000000000000 -0.1417195369im	3.92e-16

In [18]:

```
# The milestone, swept: every parameter set, n = -8 ... 8.
println("      U      β      | max rel err |G_num - G_exact|/|G_exact| over n = -8...8")
println("-"^72)
for (Ui, βi) in [(1.0,1.0), (2.0,5.0), (4.0,0.3), (0.0,2.0), (8.0,10.0), (-2.0,1.5)]
    mi = HubbardAtomModel(Ui, βi); spi = spectrum(mi); opsi = operators(mi)
    worst = 0.0
    for n in -8:8
        v = fermionic_freq(n)
        g = propagator(mi, v; sp = spi, ops = opsi); e = propagator_exact(mi, v)
        worst = max(worst, abs(g - e) / abs(e))
    end
    @printf("%6.1f %6.1f      | %.3e\n", Ui, βi, worst)
end
```

U	β	max rel err G_num - G_exact / G_exact over n = -8...8
1.0	1.0	4.364e-16
2.0	5.0	3.917e-16
4.0	0.3	2.545e-16
0.0	2.0	1.962e-16
8.0	10.0	2.314e-16
-2.0	1.5	1.792e-16

Step 5 — $\ell = 4$

$\mathcal{O} = (d_\sigma, d_\sigma^\dagger, d_{\sigma'}, d_{\sigma'}^\dagger)$ over all 24 permutations. Subtract the disconnected part, Eq. (73),

$$G_{\sigma\sigma'}^{\text{dis}} = \beta G(i\omega_1)G(i\omega_3)(\delta_{\sigma\sigma'}\delta_{\omega_{23},0} - \delta_{\omega_{12},0}),$$

then amputate the external legs, Eq. (74),

$$F_{\sigma\sigma'} = \frac{G_{\sigma\sigma'}^{\text{con}}(i\omega_1, i\omega_2, i\omega_3)}{G(i\omega_1)G(-i\omega_2)G(i\omega_3)G(-i\omega_4)}, \quad \omega_4 = -\omega_{123}.$$

Appendix D does **not** enter here: its imaginary frequency shifts are needed only for the Keldysh amputation of Eq. (76), where the legs sit at real

frequencies. On the Matsubara axis the legs are exactly on discrete frequencies and there is nothing to shift.

Eq. (85) and the PDF text layer

The published Eq. (85) is correct. What is broken is the PDF *text layer*: it puts both products of the anomalous term on one line and loses the fraction bar. Extracted literally it reads

$$\beta u^2 \frac{\delta_{\omega_{12}} \text{th} + \delta_{\omega_{13}} (\text{th} - 1) + \delta_{\omega_{14}} (\text{th} + 1)}{\prod_i (i\omega_i + u) \prod_i (i\omega_i)}$$

(dimension energy⁻⁷: cannot be right)

since βu^2 is an energy, so the remaining factor must be dimensionless while that denominator carries energy⁸. Building against that form made Step 5 fail *only* in the $\beta \delta_\omega$ terms, while matching to 10^{-16} at every generic frequency point.

The printed factor is $\prod_i (i\omega_i + u) / \prod_i (i\omega_i)$:

$$F_{\uparrow\downarrow} = 2u + u^3 \frac{\sum_i (i\omega_i)^2}{\prod_i (i\omega_i)} - \frac{6u^5}{\prod_i (i\omega_i)} + \beta u^2 [\delta_{\omega_{12}} \text{th} + \delta_{\omega_{13}} (\text{th} - 1) + \delta_{\omega_{14}} (\text{th} + 1)] \frac{\prod_i (i\omega_i + u)}{\prod_i (i\omega_i)},$$

$$F_{\uparrow\uparrow} = \beta u^2 (\delta_{\omega_{14}} - \delta_{\omega_{12}}) \frac{\prod_i (i\omega_i + u)}{\prod_i (i\omega_i)},$$

with $u = U/2$, $\text{th} = \tanh(\beta u/2)$. This was first reconstructed from three independent arguments — dimensional consistency; Appendix E's separately stated second-order expansion $F_{\uparrow\downarrow} = U + \frac{1}{4}\beta U^2(\delta_{\omega_{14}} - \delta_{\omega_{13}})$, $F_{\uparrow\uparrow} = \frac{1}{4}\beta U^2(\delta_{\omega_{14}} - \delta_{\omega_{12}})$; and a brute-force τ -integration of Eq. (71) that reproduces the computed $G^{(4)}$ independently of this whole machinery — and then **confirmed by rendering p. 16 to an image and reading the printed equation**. Eqs. (28), (39), (45), (46), (73), (74) and the Appendix E results were checked the same way; only Eq. (85) was affected.

Note that whenever any δ fires the frequencies are $\pm$ -paired, $(\nu, -\nu, \nu', -\nu')$, so $\prod_i (i\omega_i + u) = \prod_i (i\omega_i - u)$ and the sign of u in that product is not observable.

In [19]:

```
# Both channels at a handful of points: generic ones, and ones where a δ fires.
println(" n1 n2 n3 | δ12 δ13 δ14 |  F↑↓ computed  F↑↓ exact  |  F↑↑ computed  F↑↑ exact")
println("-"^104)
for (n1, n2, n3) in [(0,0,0), (3,0,-2), (1,-1,0), (1,-2,0), (0,0,-1), (0,-1,0)]
    ms = vertex_frequencies(n1, n2, n3)
```

```

d(k) = ms[1].m + ms[k].m == 0 ? 1 : 0
fud, eud = vertex(model, :up, :dn, ms; sp=sp, ops=ops), vertex_exact(model, :up
dn, ms)
fuu, euu = vertex(model, :up, :up, ms; sp=sp, ops=ops), vertex_exact(model, :up
up, ms)
@printf("%3d%3d%3d | %3d%4d%4d | %13.6f %13.6f | %13.6f %13.6f\n",
n1,n2,n3, d(2),d(3),d(4), real(fud),real(eud), real(fuu),real(euu))
end

```

n1	n2	n3	612	613	614	F↑ computed	F↑ exact	F↑↑ computed	F↑↑ exact
0	0	0	0	0	0	24.964596	24.964596	-0.000000	-0.000000
3	0	-2	0	0	0	-0.393066	-0.393066	0.000000	0.000000
1	-1	0	0	0	1	37.064469	37.064469	22.636964	22.636964
1	-2	0	1	0	0	14.427505	14.427505	-22.636964	-22.636964
0	0	-1	0	1	1	76.522586	76.522586	62.411490	62.411490
0	-1	0	1	0	1	138.934076	138.934076	0.000000	0.000000

In [20]:

```

# Swept over n1,n2,n3 ∈ -3:3 (343 points). Where F is identically zero we
# require it numerically, measured against the scale of F on the grid.
println("    U      β | chan | max rel err (|F|>0) | max |F_num| where F ≡ 0 | sca
le of F")
println("-"^90)
for (Ui, βi) in [(1.0,1.0), (2.0,5.0), (4.0,0.3), (8.0,10.0), (-2.0,1.5)]
    mi = HubbardAtomModel(Ui, βi); spi = spectrum(mi); opsi = operators(mi)
    for (sa, sb, tag) in [(:up,:dn,:updn), (:up,:up,:upup)]
        re = 0.0; ze = 0.0; scale = 0.0
        for n1 in -3:3, n2 in -3:3, n3 in -3:3
            ms = vertex_frequencies(n1, n2, n3)
            got = vertex(mi, sa, sb, ms; sp=spi, ops=opsi)
            want = vertex_exact(mi, tag, ms)
            scale = max(scale, abs(want))
            abs(want) > 1e-6 ? (re = max(re, abs(got-want)/abs(want))) :
                               (ze = max(ze, abs(got)))
        end
        @printf("%6.1f %6.1f | %4s |      %.3e      |      %.3e      | %.3e\n",
            Ui, βi, tag, re, ze, scale)
    end
end
end

```

U	β	chan	max rel err (F >0)	max F_num where F ≡ 0	scale of F
1.0	1.0	updn	1.238e-13	0.000e+00	1.371e+00
1.0	1.0	upup	7.981e-13	1.115e-14	2.628e-01
2.0	5.0	updn	5.606e-14	0.000e+00	1.389e+02
2.0	5.0	upup	4.764e-15	3.911e-15	6.241e+01
4.0	0.3	updn	5.238e-14	0.000e+00	5.893e+00
4.0	0.3	upup	3.397e-13	7.888e-14	1.289e+00
8.0	10.0	updn	8.721e-16	0.000e+00	1.214e+07
8.0	10.0	upup	5.038e-16	1.673e-11	4.257e+06
-2.0	1.5	updn	2.131e-14	0.000e+00	5.911e+00
-2.0	1.5	upup	4.312e-14	5.746e-15	2.262e+00

Step 6 — null test

Set $U = 0$. Every connected object must vanish identically. This exercises all 24 permutations and every sign ζ_p against a target of exactly zero.

It also drives the anomalous branch of Eq. (46) hard: at $U = 0$ the whole spectrum is degenerate, so $\omega' = 0$ for every PSF term and every vanishing bosonic composite triggers the second branch. A sign error there would show up here.

In [21]:

```
println(" β      | max |G^con|          | max |F|          | max |G^dis| (nonzer
o!)")
println("-"^76)
for βi in (0.5, 2.0, 7.0)
    m0 = HubbardAtomModel(0.0, βi); sp0 = spectrum(m0); ops0 = operators(m0)
    gc = 0.0; fv = 0.0; gd = 0.0
    for n1 in -3:3, n2 in -3:3, n3 in -3:3, ch in (:up,:dn), (:up,:up))
        ms = vertex_frequencies(n1, n2, n3)
        gc = max(gc, abs(connected_4p(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
        fv = max(fv, abs(vertex(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
        gd = max(gd, abs(disconnected_4p(m0, ch[1], ch[2], ms; sp=sp0, ops=ops0)))
    end
    @printf("%5.1f | %.3e          | %.3e          | %.3e\n", βi, gc, fv, gd)
end
println("\nG^dis is large, so G^con = G - G^dis vanishing is a real cancellation,")
println("not a case of both sides being zero.")
```

β	max G^con	max F	max G^dis (nonzero!)
0.5	4.337e-19	2.737e-14	1.267e-02
2.0	2.776e-17	6.844e-15	8.106e-01
7.0	1.807e-15	1.656e-14	3.475e+01

G^dis is large, so G^con = G - G^dis vanishing is a real cancellation,
not a case of both sides being zero.

Keldysh formalism

The PSFs of Eq. (28) contain no reference to the Matsubara, Keldysh or zero-temperature formalism, so this section reuses the `S` object — and the permutation loop and `ζ_p` signs — **unchanged**, and replaces only the kernel. No new exact diagonalisation.

$$G^{[\eta_1 \dots \eta_\alpha]}(\omega) = \frac{2}{2^{\ell/2}} \sum_p \zeta^p \int d^{\ell-1} \omega'_p K^{[\hat{\eta}_1 \dots \hat{\eta}_\alpha]}(\omega_p - \omega'_p) S[\mathcal{O}_p](\omega'_p) \quad (67a)$$

$$K^{[\hat{\eta}_1 \dots \hat{\eta}_\alpha]}(\omega_p - \omega'_p) = \sum_{j=1}^{\alpha} (-1)^{j-1} \prod_{i=1}^{\ell-1} \frac{1}{\omega_{1 \dots \bar{i}}^{[\hat{\eta}_j]} - \omega'_{1 \dots \bar{i}}} \quad (67b)$$

Conventions, all read from the printed page (the text layer garbles several):

- A Keldysh component $\mathbf{k} = k_1 \dots k_\ell$, $k_i \in \{1, 2\}$, is the ordered list of the slots of its 2's: $1111 = []$, $2121 = [13]$.
- $\omega_i^{[\eta]} = \omega_i + i(\ell - 1)\gamma_0$ for $i = \eta$, and $\omega_i - i\gamma_0$ otherwise (Eq. 49, p. 10).

- A 2 in slot η_j moves to slot $\mu_j = p^{-1}(\eta_j)$ — the **inverse**, which the text layer prints as "p-1" (p. 13). Then $[\hat{\eta}_1 \dots \hat{\eta}_\alpha] = \text{sort}\{\mu_j\}$ and $\bar{\hat{\eta}}_j = p(\hat{\eta}_j)$.
- $\gamma_0 > 0$ is a required argument, never a module constant.

Not implemented: Eq. (59b). Its last line has both imaginary-part signs flipped relative to the $\gamma^{[\eta]}$ convention — as printed, $\Delta(\omega) \rightarrow +2\pi i \delta(\omega_1 \dots \omega_\lambda)$, not the $-2\pi i \delta$ the text states. The route through Eqs. (67) bypasses it entirely.

In [22]:

```
# K1 -- complex tuples w^[η]: the positive imaginary part sits in slot η, and they
sum to zero
w = [0.3, -1.1, 0.7, 0.1]
for η in 1:4
    z = omega_eta(w, η, 1e-3)
    @printf("η=%d Im w^[η] = %s    |Σ w^[η]| = %.1e\n", η, string(round.(imag(z), d
igits=4)), abs(sum(z)))
end

# p. 10: the composite w^[η]_{1̄...ī} has Im < 0 for i < μ and Im > 0 for μ ≤ i < ℓ, μ
= p-1(η)
ok = true
for η in 1:4, p in all_permutations(4)
    z = omega_eta(w, η, 1e-3); μ = invperm(p)[η]; acc = 0.0im
    for i in 1:3
        acc += z[p[i]]
        global ok &= (i < μ ? imag(acc) < 0 : imag(acc) > 0)
    end
end
println("\nsign pattern of the composites, all η and all 24 p: ", ok)
```

```
η=1 Im w^[η] = [0.003, -0.001, -0.001, -0.001]    |Σ w^[η]| = 8.3e-17
η=2 Im w^[η] = [-0.001, 0.003, -0.001, -0.001]    |Σ w^[η]| = 8.3e-17
η=3 Im w^[η] = [-0.001, -0.001, 0.003, -0.001]    |Σ w^[η]| = 8.3e-17
η=4 Im w^[η] = [-0.001, -0.001, -0.001, 0.003]    |Σ w^[η]| = 8.3e-17
```

sign pattern of the composites, all η and all 24 p: true

In [23]:

```
# K2 -- Keldysh index bookkeeping. The worked example of p. 13, intermediate includ
ed.
khat, ηbh, μ = permuted_keldysh([2, 4], [4, 1, 2, 3])
println("k = 1212 = [24], p = (4123)")
println(" [μ1μ2] = ", μ, " (paper: [31])")
println(" k_p = ", khat, " = ", keldysh_label(khat, 4), " (paper: 2121 = [1
3])")
println(" η̂ = ", ηbh, " (external labels, a permutation of [2,4])")

# The example alone is weak: p and p-1 both map {2,4} onto {1,3}, so the WRONG rule
# p(ηj) also yields k_p = [13]. Only [μ1μ2] separates them. The decisive check is
# against a convention-free reference -- permute the digits, k_p = (k_{p(1)}, ..., k_
{p(ℓ)}):
println("\nℓ p-1 rule vs digit permutation p rule would be wrong in")
for ℓ in 2:4
    right = 0; wrong = 0; total = 0
    for k in all_keldysh_components(ℓ), p in all_permutations(ℓ)
        ref = keldysh_slots([keldysh_digits(k, ℓ)[p[i]] for i in 1:ℓ])
        right += permuted_keldysh(k, p)[1] == ref
        wrong += sort([p[e] for e in k]) != ref
        total += 1
    end
end
```

```

    @printf(" %d          %3d / %3d          %3d / %3d\n", l, right, total,
l, wrong, total)
end
println("\nEvery  $\ell = 2$  permutation is an involution, so  $\ell = 2$  can never catch a  $p/p^{-1}$  error.")

```

```

k = 1212 = [24],  p = (4123)
[ $\mu_1\mu_2$ ] =
[3, 1]          (paper: [31])
k_p            =
[1, 3] = 2121  (paper: 2121 = [13])
 $\eta^*$           = [4, 2]          (external labels, a permutation of [2,4])

 $\ell$    $p^{-1}$  rule vs digit permutation      p rule would be wrong in
2      8 / 8                               0 / 8
3      48 / 48                             12 / 48
4      384 / 384                           168 / 384

```

Every $\ell = 2$ permutation is an involution, so $\ell = 2$ can never catch a p/p^{-1} error.

In [24]:

```

# K3 -- the kernels.  $\alpha = 1$  must recover Eq. (52), whose label is a PERMUTED slot, s
o the
# retarded kernel is called with  $\eta_{\text{perm}} = p^{-1}(\eta_{\text{ext}})$ . And Eq. (67b) is checked agai
nst an
# independent route, Eq. (63) + Eq. (52), which reads permuted digits and uses no o
rdered lists.
worst52 = 0.0; worst63 = 0.0; n63 = 0
for  $\ell$  in 2:4, p in all_permutations( $\ell$ )
    wv = [-0.9 + 0.8i for i in 1: $\ell$ ]; wv[end] = -sum(wv[1:end-1]); v = [0.3 - 0.35i f
or i in 1: $\ell$ -1]
    for  $\eta$  in 1: $\ell$ 
        global worst52 = max(worst52, abs(keldysh_kernel(wv, v, [ $\eta$ ], p, 0.02) -
retarded_kernel(wv, v, invperm(p)[ $\eta$ ], p,
0.02)))
    end
    for k in all_keldysh_components( $\ell$ )
        global worst63 = max(worst63, abs(keldysh_kernel(wv, v, k, p, 0.05) - keldy
sh_kernel_eq63(wv, v, k, p, 0.05)))
        global n63 += 1
    end
end
@printf("alpha = 1:  Eq. (67b) vs Eq. (52)                max |diff| = %.1e\n", w
orst52)
@printf("all alpha:  Eq. (67b) vs Eq. (63)+(52), %d ( $\ell,k,p$ )    max |diff| = %.1e\n", n6
3, worst63)

```

```

alpha = 1:  Eq. (67b) vs Eq. (52)                max |diff| = 0.0e+00
all alpha:  Eq. (67b) vs Eq. (63)+(52), 440 ( $\ell,k,p$ )    max |diff| = 0.0e+00

```

Validation V1-V3

V1 is $G^{1\cdots 1} = 0$, from $K^{[]} = 0$. **V2** reproduces Appendix C at $\ell = 2$, where the prefactor $2/2^{\ell/2}$ is exactly 1 and $G^{21} = G^{[1]} = G^R$, $G^{12} = G^{[2]} = G^A$, $G^{22} = G^{[12]} = G^K$. The FDT (C4) holds only as $\gamma_0 \rightarrow 0$ — the paper says so — so it is tested as a limit.

V3 compares fully retarded components with the Matsubara result through Eq. (69), $2^{\ell/2-1}G^{[\eta]}(\omega) = \tilde{G}(i\omega)|_{i\omega \rightarrow \omega^{[\eta]}}$. The trap: $\tilde{G}$ of Eq. (68b) is built from the

regular kernel of Eq. (42), not the full Eq. (46) kernel. `correlator` now has `kernel = :regular` for it, and `regular_sum` evaluates the same object at arbitrary complex frequencies — which is what the continuation needs.

In [25]:

```
# V1
v1 = maximum(abs(keldysh_correlator(model, 0s, wv, Int[]; γ0 = 1e-3, sp = sp, is_fermionic = f))
              for (0s, f, wv) in (((ops.d_up, ops.dag_up), [true, true], [0.4, -0.4]),
                                   ((ops.d_up, ops.n_up, ops.dag_up), [true, false, true], [0.4, -1.0, 0.6]),
                                   ((ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn), fill(true, 4), [0.4, -1.0, 0.6, 0.0])))
@printf("V1: max |G^{1...l}| over l = 2, 3, 4: %.1e\n\n", v1)

# V2 -- the Hubbard atom in closed form, exact at finite γ0
thV = tanh(β * u / 2); v2 = 0.0
for γ0 in (0.5, 1e-4), ω in (-2.3, -0.4, 0.0, 0.7)
    g = k -> keldysh_correlator(model, (ops.d_up, ops.dag_up), [ω, -ω], k; γ0 = γ0, sp = sp)
    R = 0.5 * (1/(ω + im*γ0 + u) + 1/(ω + im*γ0 - u)); A = 0.5 * (1/(ω - im*γ0 + u) + 1/(ω - im*γ0 - u))
    K = 0.5 * (-thV * (1/(ω + im*γ0 + u) - 1/(ω - im*γ0 + u)) + thV * (1/(ω + im*γ0 - u) - 1/(ω - im*γ0 - u)))
    global v2 = max(v2, abs(g([1]) - R), abs(g([2]) - A), abs(g([1, 2]) - K))
end
@printf("V2: G^R, G^A, G^K of the atom at finite γ0, max |diff| vs closed form: %.1e\n\n", v2)

println("V2: FDT (C4) at the pole ω = E, ratio G^[12]/(G^[1]-G^[2]) → tanh(βE/2) as γ0 → 0")
for E in (1.0, -1.0), γ0 in (1e-2, 1e-3, 1e-4)
    g = k -> keldysh_correlator(model, (ops.d_up, ops.dag_up), [E, -E], k; γ0 = γ0, sp = sp)
    r = g([1, 2]) / (g([1]) - g([2]))
    @printf("    E = %.0f    γ0 = %.0e    |ratio - tanh(βE/2)| = %.2e\n", E, γ0, abs(r - tanh(βE/2)))
end
println("    (a factor 100 per decade: the FDT is recovered as O(γ0^2))")
```

V1: max |G^{1...l}| over l = 2, 3, 4: 0.0e+00

V2: G^R, G^A, G^K of the atom at finite γ0, max |diff| vs closed form: 2.2e-16

V2: FDT (C4) at the pole ω = E, ratio G^[12]/(G^[1]-G^[2]) → tanh(βE/2) as γ0 → 0

E = +1	γ0 = 1e-02	ratio - tanh(βE/2) = 4.93e-05
E = +1	γ0 = 1e-03	ratio - tanh(βE/2) = 4.93e-07
E = +1	γ0 = 1e-04	ratio - tanh(βE/2) = 4.93e-09
E = -1	γ0 = 1e-02	ratio - tanh(βE/2) = 4.93e-05
E = -1	γ0 = 1e-03	ratio - tanh(βE/2) = 4.93e-07
E = -1	γ0 = 1e-04	ratio - tanh(βE/2) = 4.93e-09

(a factor 100 per decade: the FDT is recovered as O(γ0^2))

In [26]:

```
# V3 -- fully retarded components against the continued regular MF sum, Eq. (69)
println("V3: 2^{l/2-1} G^[η](ω) vs G-tilde continued to ω^[η]")
for (name, 0s, f) in [("l=2 (d↑,d†↑)", (ops.d_up, ops.dag_up), [true, true]),
                     ("l=3 (d↑,n↑,d†↑)", (ops.d_up, ops.n_up, ops.dag_up), [true, false, true]),
                     ("l=4 ↑↓", (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn), fill(true, 4))]
    l = length(0s); c = permuted_psfs(sp, 0s; is_fermionic = f); worst = 0.0; scale
```

```

= 0.0
  for s in 1:6, γ0 in (0.2, 1e-3), η in 1:ℓ
    wv = [2sin(0.9s + 1.3i) for i in 1:ℓ]; wv[end] = -sum(wv[1:end-1])
    kf = keldysh_correlator(model, 0s, wv, [η]; γ0 = γ0, sp = sp, cache = c, is_
    _fermionic = f)
    mf = regular_sum(sp, 0s, omega_eta(wv, η, γ0); cache = c, is_fermionic = f)
    worst = max(worst, abs(2.0^(ℓ/2 - 1) * kf - mf)); scale = max(scale, abs(m
    f))
  end
  @printf("    %-16s max |diff| = %.1e    (scale %.1e)\n", name, worst, scale)
end

# The ↑↑ fully retarded components vanish identically: F↑↑ in Eq. (85b) is purely
# anomalous, so there is no regular part to continue.
Oud = (ops.d_up, ops.dag_up, ops.d_dn, ops.dag_dn); Ouu = (ops.d_up, ops.dag_up, op
s.d_up, ops.dag_up)
cud = permuted_psfs(sp, Oud); cuu = permuted_psfs(sp, Ouu); wv = [0.9, -0.3, 1.4, -
2.0]
@printf("\n    |G^[η]| at a generic point:    ↑↑ %.2e    ↑↑ %.2e\n",
        maximum(abs(keldysh_correlator(model, Oud, wv, [η]; γ0 = 1e-3, sp = sp, cac
he = cud)) for η in 1:4),
        maximum(abs(keldysh_correlator(model, Ouu, wv, [η]; γ0 = 1e-3, sp = sp, cac
he = cuu)) for η in 1:4))

```

V3: $2^{\{\ell/2-1\}} G^{\eta}(\omega)$ vs $\tilde{G}$ continued to ω^{η}

ℓ=2 (d↑,d↑↑)	max diff = 0.0e+00	(scale 6.8e+00)
ℓ=3 (d↑,n↑,d↑↑)	max diff = 1.8e-15	(scale 9.5e+00)
ℓ=4 ↑↓	max diff = 0.0e+00	(scale 1.8e+01)

|G^[η]| at a generic point: ↑↓ 2.38e+00 ↑↑ 3.11e-15

V4 and V6 — all 16 components, and where the anomalous weight goes

For each component, $|G^k|$ is compared at $\gamma_0 = 10^{-4}$ and 10^{-6} : it is **finite** (converges), **→0** (vanishes linearly), or a **δ** (grows like $1/\gamma_0$ — a delta function in the limit). Four frequency points: a generic one, and one on each bosonic hyperplane $\omega_{12} = 0$, $\omega_{13} = 0$, $\omega_{14} = 0$, where the MF vertex (85) has its $\beta u^2 \delta_\omega$ terms. The connected part is used, split on the PSFs by Eq. (31) — validated above against the MF's Eq. (73) route.

One pitfall is guarded against: a component that is **exactly zero** but computed as a cancelling sum of $1/\gamma_0$ -sized terms leaves a roundoff residue that grows like a divergence. The ↑↑ fully retarded components do exactly this, with noisy non-integer "slopes"; they are classified as zero.

In [27]:

```

Gk(c, wv, k, g, K = keldysh_kernel) = 0.5 * sum(d.ζ * K(wv, t.position, k, d.p, g)
* t.weight

                                for d in c for t in d.terms; init =
0.0im)
function scaling(c, wv, k, K = keldysh_kernel)
  a = abs(Gk(c, wv, k, 1e-4, K)); b = abs(Gk(c, wv, k, 1e-6, K))
  max(a, b) < 1e-6 && return "0"
  s = (log10(b) - log10(a)) / (-2)
  s < -0.5 ? "δ" : s > 0.5 ? "→0" : "finite"
end
pts = [("generic", [0.37, -1.13, 0.71, 0.05]), ("ω12=0", [0.37, -0.37, 0.71, -0.7

```

```

1]),
    ("ω13=0", [0.37, 0.71, -0.37, -0.71]), ("ω14=0", [0.37, 0.71, -0.71, -0.3
7]))
tables = Dict{String,Any}()
for (chan, Os) in [("↑↓", Oud), ("↑↑", Ouu)]
    c = permuted_psfs(sp, Os; part = :connected)
    println("\n", chan, "    connected    (U = $U, β = $β)")
    @printf("    k    α    | %-8s| %-8s| %-8s| %-8s\n", [n for (n, _) in pts]...)
    for k in all_keldysh_components(4)
        row = [scaling(c, wv, k) for (_, wv) in pts]; tables[chan * keldysh_label
(k, 4)] = row
        @printf("    %s    %d    | %-8s| %-8s| %-8s| %-8s\n", keldysh_label(k, 4), length
h(k), row...)
    end
end
end

```

```

↑↓  connected    (U = 2.0, β = 5.0)
k    α    | generic | ω12=0 | ω13=0 | ω14=0
1111 0    | 0          | 0      | 0      | 0
2111 1    | finite     | finite  | finite  | finite
1211 1    | finite     | finite  | finite  | finite
1121 1    | finite     | finite  | finite  | finite
1112 1    | finite     | finite  | finite  | finite
2211 2    | →0         | →0      | δ       | δ
2121 2    | →0         | δ       | →0      | δ
2112 2    | →0         | δ       | δ       | →0
1221 2    | →0         | δ       | δ       | →0
1212 2    | →0         | δ       | →0      | δ
1122 2    | →0         | →0      | δ       | δ
2221 3    | finite     | finite  | finite  | finite
2212 3    | finite     | finite  | finite  | finite
2122 3    | finite     | finite  | finite  | finite
1222 3    | finite     | finite  | finite  | finite
2222 4    | →0         | →0      | →0      | →0

```

```

↑↑  connected    (U = 2.0, β = 5.0)
k    α    | generic | ω12=0 | ω13=0 | ω14=0
1111 0    | 0          | 0      | 0      | 0
2111 1    | 0          | 0      | 0      | 0
1211 1    | 0          | 0      | 0      | 0
1121 1    | 0          | 0      | 0      | 0
1112 1    | 0          | 0      | 0      | 0
2211 2    | →0         | →0      | →0      | δ
2121 2    | →0         | δ       | →0      | δ
2112 2    | →0         | δ       | →0      | →0
1221 2    | →0         | δ       | →0      | →0
1212 2    | →0         | δ       | →0      | δ
1122 2    | →0         | →0      | →0      | δ
2221 3    | →0         | finite  | →0      | finite
2212 3    | →0         | finite  | →0      | finite
2122 3    | →0         | finite  | →0      | finite
1222 3    | →0         | finite  | →0      | finite
2222 4    | →0         | →0      | →0      | →0

```

Reading the table.

- **Fully retarded components ($\alpha = 1$) never carry a δ** , at any point — as p. 14 says, the $\beta\delta_\omega$ structures "cannot appear in a fully retarded component".
- In $\uparrow\downarrow$, at $\omega_{ab} = 0$ the δ sits in exactly the four $\alpha = 2$ components with **one 2 inside $\{a, b\}$ and one outside**; the other two vanish. The rule holds on all three hyperplanes.

- In $\uparrow\uparrow$, **nothing carries a δ at $\omega_{13} = 0$** — and Eq. (85b) indeed has no $\delta_{\omega_{13}}$ term, while (85a) has all three. The Keldysh code reproduces which δ 's are absent without being told.

In [28]:

```
# V6, quantitatively: the  $\delta$  weight  $c = \lim \gamma_0 \cdot G^{\text{con},k}$  scales with  $U$  like the anomalous
# coefficients of Eq. (85). At small  $U$ ,  $th = \tanh(\beta u/2) \approx \beta u/2$ , so the  $\uparrow\downarrow \delta_{\{\omega_{12}\}}$  term,
#  $\propto \beta u^2 \cdot th$ , carries one power of  $U$  MORE than the others.
function  $\delta$ weight_exponent(0s, wv, k)
    cs = map((0.08, 0.02)) do Ui
        c = permuted_psfs(spectrum(HubbardAtomModel(Ui, 5.0)), 0s; part = :connected)
        abs(1e-7 * Gk(c, wv, k, 1e-7))
    end
    (log10(cs[2]) - log10(cs[1])) / (log10(0.02) - log10(0.08))
end
println("chan  $\delta$  at k Eq. (85) coefficient predicted measured")
println("-"^72)
for (chan, 0s, name, idx, k, coef, pred) in [
    ("↑↓", Oud, " $\omega_{12}=0$ ", 2, [1,3], " $\beta u^2 \cdot th$ ", 3), ("↑↓", Oud, " $\omega_{13}=0$ ", 3,
    [1,2], " $\beta u^2(th-1)$ ", 2),
    ("↑↓", Oud, " $\omega_{14}=0$ ", 4, [1,2], " $\beta u^2(th+1)$ ", 2), ("↑↑", Ouu, " $\omega_{12}=0$ ", 2,
    [1,3], " $-\beta u^2$ ", 2),
    ("↑↑", Ouu, " $\omega_{14}=0$ ", 4, [1,2], " $\beta u^2$ ", 2)]
    @printf(" %s %s %s %-22s U^%d U^%.3f\n", chan, name, keldysh_label(k, 4), coef, pred,
         $\delta$ weight_exponent(0s, pts[idx][2], k))
end

# Appendix E's model case at  $\ell = 2$ : a conserved density has  $G^R \equiv 0$  ( $[H, n] = 0$ ),
# while  $G^K$  carries all the weight as  $\delta(\omega)$  -- the KF face of the MF term  $\propto \beta \delta_{\{\omega, 0\}}$ .
gn = (k,  $\omega$ ,  $\gamma_0$ ) -> keldysh_correlator(model, (ops.n_up, ops.n_up), [ $\omega$ ,  $-\omega$ ], k;  $\gamma_0 = \gamma_0$ , sp = sp,
    is_fermionic = [false, false])
@printf("\nApp. E model, (n↑, n↑): |G^R(0)| = %.1e |G^K(0)| at  $\gamma_0 = 1e-3, 1e-5$ :
%.3e, %.3e ( $\propto 1/\gamma_0$ )\n",
    abs(gn([1], 0.0, 1e-5)), abs(gn([1,2], 0.0, 1e-3)), abs(gn([1,2], 0.0, 1e-5)))
```

chan	δ at	k	Eq. (85) coefficient	predicted	measured
↑↓	$\omega_{12}=0$	2121	$\beta u^2 \cdot th$	U^3	$U^{3.008}$
↑↓	$\omega_{13}=0$	2211	$\beta u^2(th-1)$	U^2	$U^{1.953}$
↑↓	$\omega_{14}=0$	2211	$\beta u^2(th+1)$	U^2	$U^{2.061}$
↑↑	$\omega_{12}=0$	2121	$-\beta u^2$	U^2	$U^{2.010}$
↑↑	$\omega_{14}=0$	2211	βu^2	U^2	$U^{2.010}$

App. E model, (n↑, n↑): |G^R(0)| = 6.8e-13 |G^K(0)| at $\gamma_0 = 1e-3, 1e-5$: 2.000e+03, 2.000e+05 ($\propto 1/\gamma_0$)

In [29]:

```
# The trap the task warns about: pairing the EXTERNAL labels  $\eta_j$  with  $(-1)^{j-1}$  in
# external order, instead of the permuted order. Exact for  $\alpha \leq 1$ , so V1-V3 cannot see it.
P(z, p, wp) = (acc = 0.0im; K = 1.0 + 0im; for i in 1:length(z)-1; acc += z[p[i]];
K /= (acc - wp[i]); end; K)
extorder( $\omega$ , wp, k, p, g) = isempty(k) ? 0.0im : sum((-1)^(j-1) * P(omega_eta( $\omega$ , e,
g), p, wp) for (j, e) in enumerate(k))
c = permuted_psfs(sp, Ouu; part = :connected)
 $\alpha_1$  = maximum(abs(Gk(c, pts[1][2], k, 1e-3) - Gk(c, pts[1][2], k, 1e-3, extorder)))
```

```

        for k in all_keldysh_components(4) if length(k) <= 1)
nd = sum(scoring(c, wv, k) != scoring(c, wv, k, extorder) for k in all_keldysh_comp
onents(4) for (_, wv) in pts)
@printf("trap kernel,  $\alpha \leq 1$  components: max |diff| = %.1e (invisible to V3)\n",
 $\alpha$ )
@printf("trap kernel, V4 table ( $\uparrow\uparrow$ ): %d of 64 classifications change (V4 catc
hes it)\n", nd)

```

```

trap kernel,  $\alpha \leq 1$  components: max |diff| = 0.0e+00 (invisible to V3)
trap kernel, V4 table ( $\uparrow\uparrow$ ): 44 of 64 classifications change (V4 catches it)

```

V5 — the $U = 0$ null test

At $U = 0$ Wick's theorem makes $S - S^{\text{dis}}$ vanish as a measure, so every connected Keldysh component vanishes identically, at generic and δ points alike.

A caveat, found while building this. Because the split is done on the PSFs, that cancellation happens *permutation by permutation* — it does not by itself exercise ζ_p or the K2 bookkeeping. The tempting alternative, "the full G vanishes at generic points as $\gamma_0 \rightarrow 0$ ", is no better: any alternating-sign kernel reduces there to the regular sum, which is identically zero at $U = 0$, so every bookkeeping bug tested also passes — and the trap kernel above even gives *exactly* zero. The checks with real teeth for ζ_p and K2 are V3, the Eq. (63) identity, and the V4 table.

In [30]:

```

m0 = HubbardAtomModel(0.0, 2.0); sp0 = spectrum(m0); o0 = operators(m0)
for (chan, 0s) in [(" $\uparrow\downarrow$ ", (o0.d_up, o0.dag_up, o0.d_dn, o0.dag_dn)), (" $\uparrow\uparrow$ ", (o0.d_u
p, o0.dag_up, o0.d_up, o0.dag_up))]
    con = permuted_psfs(sp0, 0s; part = :connected)
    sw = maximum(maximum(abs(t.weight) for t in aggregate_psf(d.terms); init = 0.0)
for d in con)
    gc = maximum(abs(keldysh_correlator(m0, 0s, wv, k;  $\gamma_0$  = g, sp = sp0, cache = co
n))
        for (_, wv) in pts, k in all_keldysh_components(4), g in (1e-1, 1e
-4))
    @printf("%s: max | $S^{\text{con}}$ | weight over 24 p = %.1e max | $G^{\text{con},k}$ | over 16 k, 4
points, 2  $\gamma_0$  = %.1e\n", chan, sw, gc)
end
full = permuted_psfs(sp0, (o0.d_up, o0.dag_up, o0.d_dn, o0.dag_dn))
@printf("\nfull G at the generic point, trap kernel: max | $G^k$ | = %.1e <- why V5 a
lone has no teeth for K2\n",
    maximum(abs(Gk(full, pts[1][2], k, 1e-4, extorder)) for k in all_keldysh_co
mponents(4)))

```

```

 $\uparrow\downarrow$ : max | $S^{\text{con}}$ | weight over 24 p = 0.0e+00 max | $G^{\text{con},k}$ | over 16 k, 4 points, 2  $\gamma_0$  = 1.8e-12
 $\uparrow\uparrow$ : max | $S^{\text{con}}$ | weight over 24 p = 0.0e+00 max | $G^{\text{con},k}$ | over 16 k, 4 points, 2  $\gamma_0$  = 7.3e-12
full G at the generic point, trap kernel: max | $G^k$ | = 2.1e-14 <- why V5 alone has no teeth for K2

```